

Ottimizzazione dello stoccaggio energetico tramite idrogeno di un impianto eolico utilizzando Reinforcement Learning

Davide Sarti

Febbraio - Marzo 2025

Contents

1	Introduzione	3
1.1	Brevi cenni su stato dell'arte	3
1.2	Obiettivi	3
1.3	Tecnologie utilizzate	3
2	Struttura dell'ambiente	3
2.1	Descrizione dei componenti	3
2.2	Class diagram	4
2.3	Spazio degli stati e delle azioni	4
3	Preprocessing dei dati e rete neurale	4
3.1	Dati di un sistema eolico	4
3.2	Rete per la previsione della potenza	5
3.3	Risultati	6
4	Modello di RL	6
4.1	Studio della reward	6
4.2	Integrazione del modello predittivo	7
4.3	Training e iperparametri	8
5	Risultati e conclusioni	9
5.1	Risultati del modello	9
5.2	Approccio naive	10
5.3	Confronto tra i due approcci	12
5.4	Miglioramenti e sviluppi futuri	12

1 Introduzione

1.1 Brevi cenni su stato dell'arte

I trend attuali nell'uso dell'AI nelle smart grid includono l'ottimizzazione della gestione della domanda e dell'offerta energetica, la manutenzione predittiva delle infrastrutture e il miglioramento della resilienza della rete tramite algoritmi avanzati di machine learning. Il reinforcement learning gioca un ruolo chiave nell'adattamento dinamico dei sistemi di controllo, migliorando l'efficienza operativa e la risposta alle variazioni della rete attraverso l'apprendimento basato sull'esperienza. Una delle varie applicazioni prevede lo stoccaggio intelligente dell'energia sfruttando modelli che ottimizzano il caricamento e il rilascio dell'energia accumulata, riducendo sprechi e migliorando l'integrazione delle fonti rinnovabili nella rete.

1.2 Obiettivi

L'obiettivo del progetto è lo sviluppo di un modello di reinforcement learning in grado di stabilizzare al di sopra di un target l'energia eolica prodotta da un impianto tramite l'utilizzo di idrogeno come mezzo di stoccaggio energetico. Il sistema include anche un secondo modello dedicato all'approssimazione della relazione tra il vento e la produzione di energia dell'impianto.

La rappresentazione del sistema è di forte carattere matematico, ampliabile successivamente considerando maggiormente gli aspetti fisici del contesto del problema.

1.3 Tecnologie utilizzate

Il progetto è scritto interamente in Python sfruttando librerie tradizionali come Numpy, Matplotlib e Sklearn per la manipolazione dei dati, le operazioni, il plot dei grafici e l'utilizzo delle metriche. Il modello approssimatore è in Pytorch mentre il modello di reinforcement learning è il PPO fornito da Stablebaselines3. La modellazione dell'environment è avvenuta con Gym.

Il training è avvenuto sulla CPU locale (AMD Ryzen™ 7 7735U) utilizzando CUDA.

2 Struttura dell'ambiente

2.1 Descrizione dei componenti

L'ambiente consiste in una composizione di classi per ogni modulo che compone la rete energetica. Si hanno 5 componenti principali:

- la sorgente energetica che determina in ogni stato la potenza in ingresso
- l'elettrolizzatore che fornisce l'interfaccia di passaggio da energia elettrica ad idrogeno
- la tanica di idrogeno
- la cella a combustibile, che fornisce l'interfaccia di passaggio da idrogeno ad energia elettrica
- l'output, che contiene uno store dell'output dello stato precedente per poter fare considerazioni sulla stabilità della potenza in uscita

2.2 Class diagram

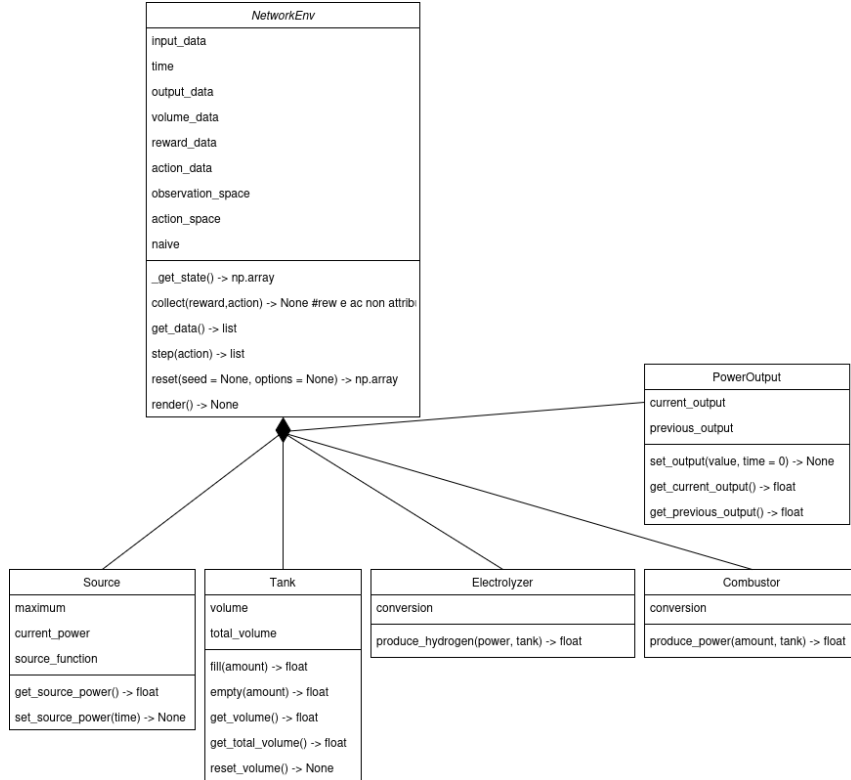


Figure 1: Class diagram dell'environment

2.3 Spazio degli stati e delle azioni

Sia azioni che stati sono rappresentati nel continuo e normalizzate tra -1 e 1, in particolare:

- Ogni stato è identificato dall'energia in entrata, mappata tra 0 e un valore indicativo superiore all'energia massima che la sorgente può fornire, e il volume della tanica, mappata tra 0 e il volume massimo della tanica.
- Ogni azione è identificata dalla percentuale di potenza da convertire in idrogeno e dalla percentuale di idrogeno da convertire in potenza, entrambe mappate tra 0 e 1.

3 Preprocessing dei dati e rete neurale

3.1 Dati di un sistema eolico

I dati relativi al sistema eolico sono stati recuperati da un progetto di un LSTM per la predizione a breve termine della potenza generata da un impianto eolico (Github). Il numero dei campioni è 50531 e le misure presentate per ogni sample del dataset sono:

- Date/Time
- LV Active Power (kW)
- Wind Speed (m/s)
- Theoretical Power Curve (KWh)
- Wind Direction (°)

La potenza teorica rispetto a quella effettiva non considera limiti e caratteristiche strutturali dell'impianto. Su un'applicazione reale, è da valutare quale delle due quantità correlare alla velocità del vento a seconda dello scopo per cui il modello viene concepito. Nel contesto corrente, si è utilizzata LV Active Power come misura indicativa della potenza generata. Viene ignorata la direzione del vento nel modello predittivo in quanto presenta bassa correlazione con la potenza generata nel dataset.



Figure 2: Correlazione tra features del dataset

Per rendere il modello predittivo più accurato è stato effettuato uno smoothing della velocità del vento del dataset originale tramite l'utilizzo di una media mobile e sono stati esclusi valori negativi.

3.2 Rete per la previsione della potenza

La rete neurale è una rete densa progettata per predire la potenza prodotta dall'impianto data la velocità del vento. L'architettura della rete è composta da tre layer principali: un layer di input da un neurone, due layer nascosti da 64 e 128 neuroni e un layer di output anch'esso composto da un solo neurone. La funzione di attivazione è ReLU per tutti i layer. Il training avviene su 2000 epoche con ottimizzatore Adam e viene utilizzata MSE come loss function. L'addestramento avviene in modalità full batch data la dimensione relativamente contenuta del dataset. Il seed è fissato per garantire la riproducibilità dell'esperimento.

3.3 Risultati

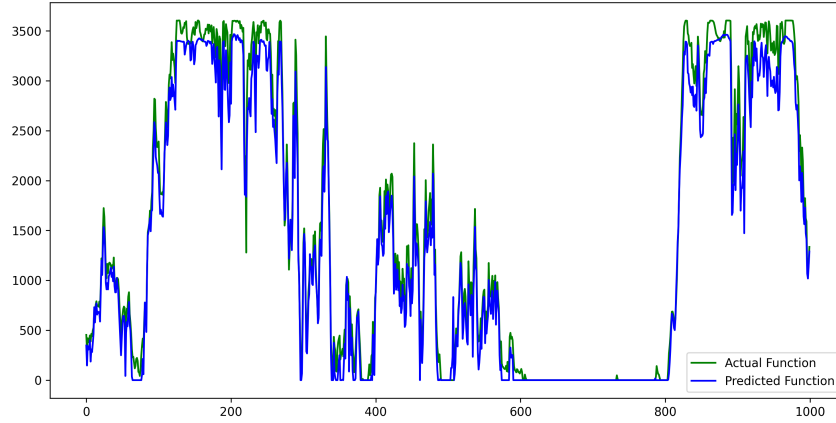


Figure 3: Potenza reale e previsione

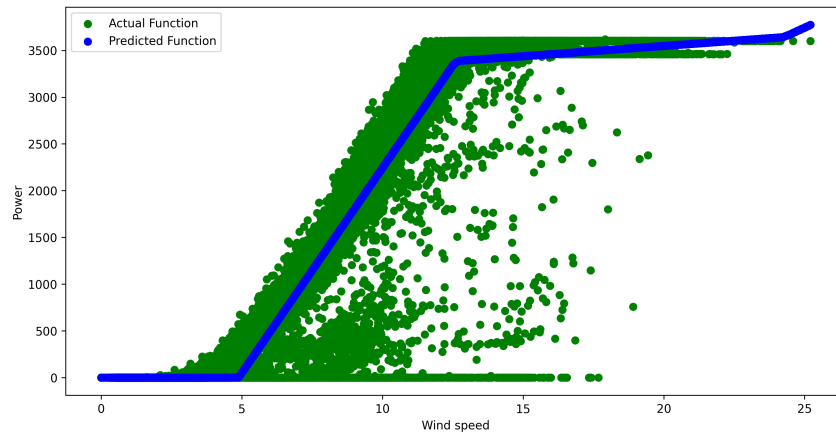


Figure 4: Potenza in funzione del vento

4 Modello di RL

4.1 Studio della reward

La funzione di reward del modello premia la vicinanza dell'output al target e il suo superamento, la stabilità della potenza in output e lo stoccaggio di idrogeno nei momenti di surplus energetico. In particolare, i vari componenti della reward sono:

- una penalizzazione quadratica della distanza dell'output dal target con offset positivo
- un premio conferito quando l'output supera il target e penalità lineare quando l'output è al di sotto di esso
- una componente di premio che si attiva quando vi è surplus energetico e contemporaneamente incentiva stoccaggio e penalizza utilizzo di idrogeno
- una penalizzazione lineare della distanza tra l'output attuale e l'output dello stato precedente

Segue l'implementazione effettiva della reward.

```

def reward_quadratic(error, scale = 25):
    return -scale * (error ** 1.5) + 5

def prevstate_reward(prevalue, value):
    if prevalue == 0:
        return 0
    error = relative_error(prevalue, value)
    return -100*error

def relative_error(target, value):
    return abs(target - value) / target

error = relative_error(TARGET_POWER, current_output)
distance = reward_quadratic(error)
bonus = -abs(TARGET_POWER - current_output)/50
if TARGET_POWER > current_output else 10
storage_bonus = power_to_h2 * 0.01 - h2_to_power * 0.02
if source_power > TARGET_POWER * 1.1 else 0
prevstate = prevstate_reward(self.output.get_previous_output(), current_output)

reward = float(distance + prevstate + bonus + storage_bonus)
reward = np.clip(reward, -30, None)

```

4.2 Integrazione del modello predittivo

Il modello predittivo per la potenza generata dal vento è chiamato nella funzione sorgente dell' environment. In particolare, per ottenere un andamento più ciclico della potenza che favorisse la convergenza dell'agente, è stata costruita una funzione periodica per modellare la variazione nel tempo del vento, a sua volta passata come input al modello predittivo.

Tale funzione periodica è stata scalata in modo da rispecchiare qualitativamente l'ordine di grandezza e l'andamento dei dati reali sulla velocità del vento.

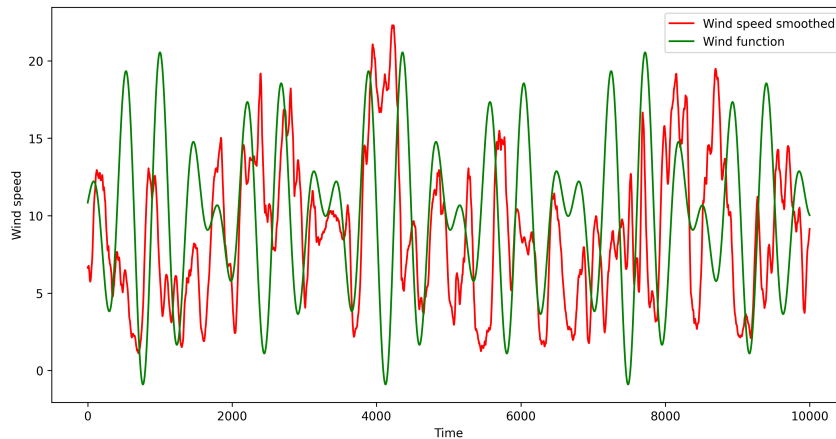


Figure 5: Confronto tra funzione periodica e dati reali sul vento

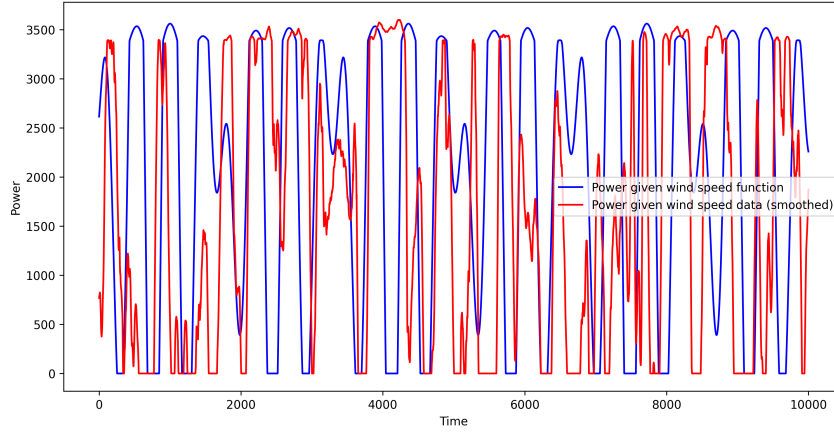


Figure 6: Potenza predetta con funzione periodica e dati reali sul vento

4.3 Training e iperparametri

Il modello allenato è il Proximal Policy Optimizer (PPO) fornito dalla libreria `stablebaselines3`. PPO è un algoritmo di reinforcement learning che ottimizza la politica agent usando un approccio basato su una funzione di perdita prossimale, che bilancia l'efficienza del miglioramento e la stabilità. Rispetto ad altri algoritmi, PPO cerca di evitare grandi aggiornamenti di politica, garantendo un miglioramento graduale e più sicuro durante l'addestramento.

Gli iperparametri sono stati assegnati con lo scopo di conferire una maggiore stabilità e robustezza all'apprendimento con particolare attenzione alla natura ciclica del problema di accumulo energetico.

```
model = PPO("MlpPolicy", env, n_epochs = 15, n_steps = 1024, batch_size = 128,
gamma = 0.995, ent_coef = 0.01, learning_rate = 1e-4, device = "cpu")
```

Il training è avvenuto sfruttando Curriculum Learning, con cicli di training multipli e salvando di tanto in tanto i modelli più performanti. Nel complesso il training si è attestato sui 10 milioni di steps, contando i modelli intermedi. Dapprima l'attenzione è stata posta su una reward che penalizzasse la distanza dal target per eliminare i picchi di potenza in eccesso. Sono stati poi aggiunti gli elementi di stabilizzazione ed incentivi ad immagazzinare idrogeno.

Nonostante il comportamento dell'agente sia coerente con gli obiettivi posti, la reward è stata fortemente oscillante anche nelle ultime fasi di training. Per quanto l'episode reward effettivamente convergesse, i plot dell'andamento della reward durante il training si sono rivelati pressoché inutili per l'estrema variabilità della reward tra un episodio e l'altro.

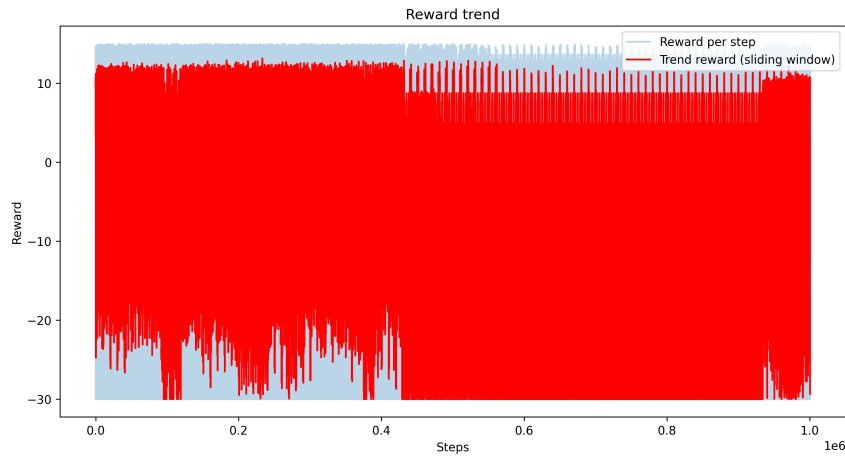


Figure 7: Reward relativa all'ultimo ciclo di training

5 Risultati e conclusioni

5.1 Risultati del modello

Per valutare le prestazioni del modello prodotto lo si è stato testato sia con la stessa sorgente d'allenamento, che utilizzando il modello predittivo con i dati di vento dal dataset al fine di valutarne le prestazioni in un ipotetico caso reale.

Si è utilizzata RMSE come metrica per la valutazione del modello, e si è confrontata la media dell'output tra i modelli.

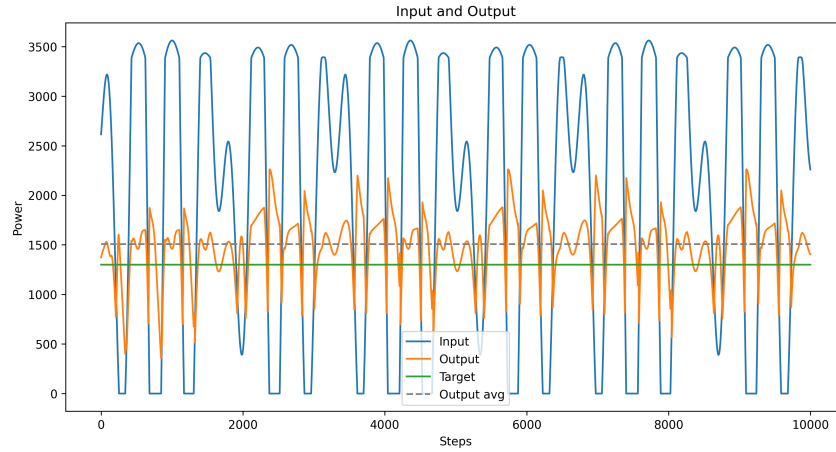


Figure 8: Confronto tra input e output nel caso simulato

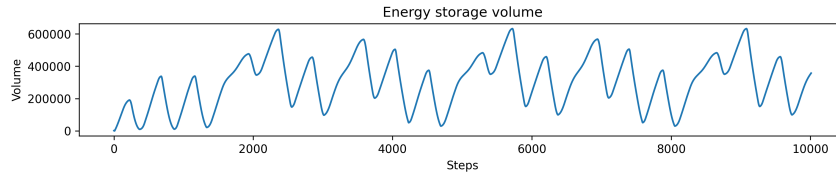


Figure 9: Utilizzo del volume nel caso simulato

RMSE: 367.9578106831468

Media: 1507.4271240234375

Nel caso simulato si ha un ambiente in cui la disponibilità di energia è in surplus rispetto al target. Per questo motivo l'aspettativa era quella di ottenere una media più alta rispetto al target.

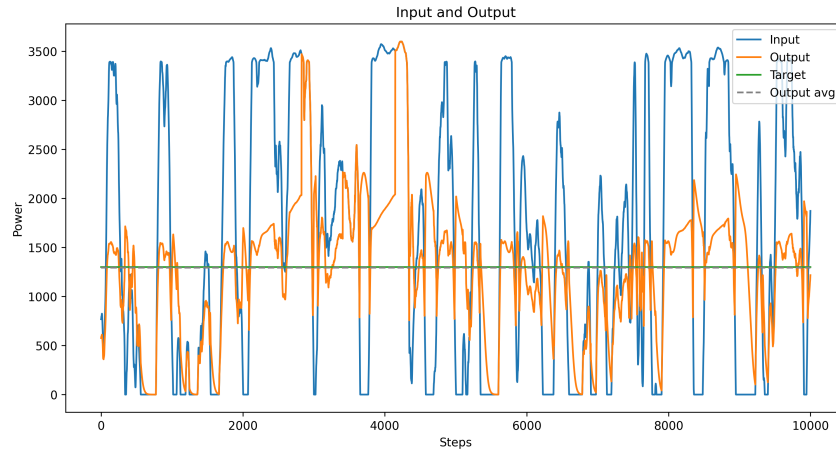


Figure 10: Confronto tra input e output nel caso reale

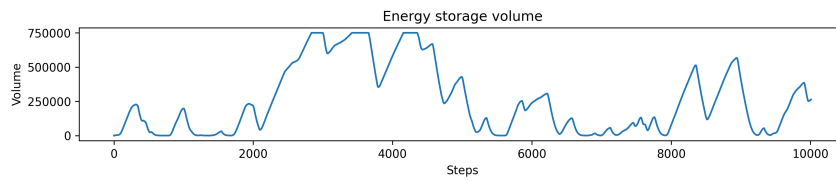


Figure 11: Utilizzo del volume nel caso reale

RMSE: 680.3080635771839

Media: 1293.820556640625

Nonostante un notevole peggioramento delle prestazioni nel caso reale, il comportamento dell'agente tende ad essere più stabile ove la potenza rispecchia un andamento più simile al caso simulato.

5.2 Approccio naive

Si è voluta introdurre una politica di gestione dell'idrogeno alternativa che non coinvolgesse l'agente per studiare la spendibilità di quest'ultimo nel contesto del problema. La politica di gestione della potenza ad ogni stato è definita come segue:

- Se la potenza della sorgente supera il target, viene prodotto idrogeno per tutta la potenza in eccesso rispetto al target
- Se la potenza della sorgente è al di sotto del target, viene prodotta potenza dall'idrogeno per raggiungere il target, tenendo conto dei tassi di conversione

Si riportano i risultati ottenuti con l'approccio naive in entrambi i tipi di sorgente, come sopra.

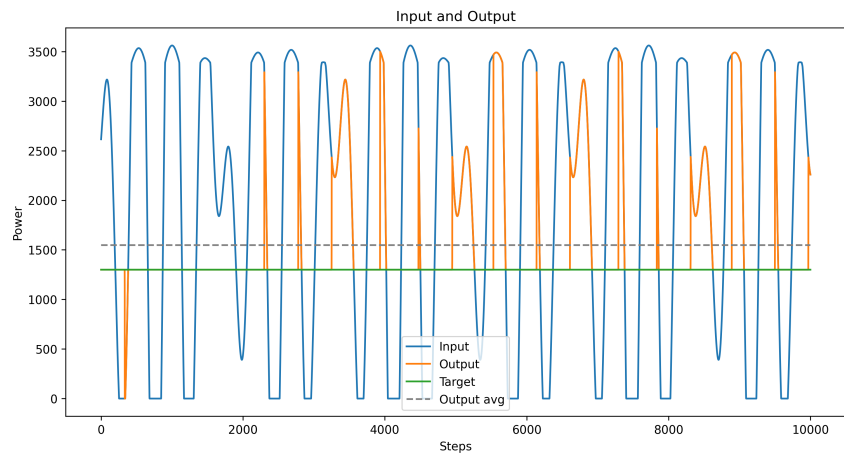


Figure 12: Approccio naive: confronto tra input e output nel caso simulato

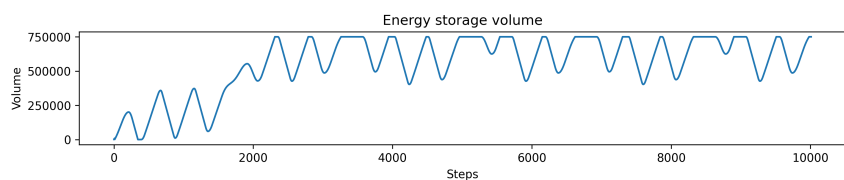


Figure 13: Approccio naive: utilizzo del volume nel caso simulato

RMSE: 624.9416106937604

Media: 1547.6804925443093

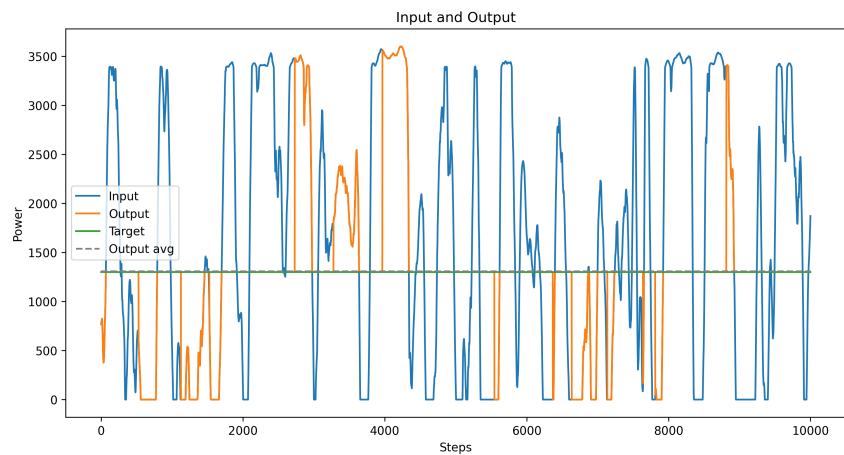


Figure 14: Approccio naive: confronto tra input e output nel caso reale

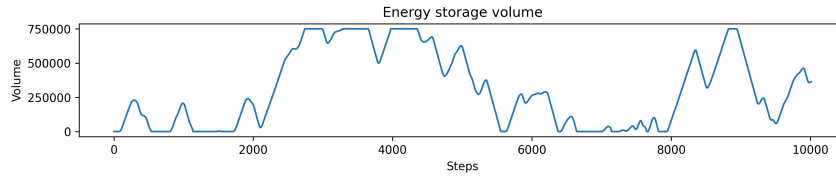


Figure 15: Approccio naive: utilizzo del volume nel caso reale

RMSE: 715.856645680191

Media: 1307.6496984705714

Sia che si utilizzino dati simulati che dati reali, l'approccio naive mostra una limitazione ricorrente. Qualora non vi sia sufficiente idrogeno nella tanica a colmare la domanda o al contrario, la tanica sia piena in un momento di surplus energetico, l'output segue esattamente la curva dell'input, fallendo completamente nella gestione della potenza del sistema. Tuttavia, quando lo stato del sistema è vantaggioso, l'approccio naive è infallibile, stabilizzando la potenza in uscita esattamente sul target.

Di conseguenza, l'approccio naive è consigliabile solamente nei casi in cui la configurazione dell'ambiente presenti un andamento dell'offerta particolarmente stabile e con una media prossima al target.

5.3 Confronto tra i due approcci

Confrontando i risultati dei due approcci differenti, si può notare come nel caso simulato le performance del modello superino quelle dell'approccio algoritmico, con un errore quasi dimezzato. Il margine è nettamente inferiore per quanto riguarda il caso reale, dove nonostante l'errore minore sia sempre attribuibile al modello, è tangibile la difficoltà dell'agente in contesti in cui la sorgente sia scarsa per un numero elevato di steps consecutivi, o al contrario troppo abbondante. Nel caso medio, tuttavia, l'andamento dell'output è coerente con le performance del caso simulato, mostrando una buona risposta del modello nel passaggio da dati simulati a dati reali.

In totale assenza di modalità di gestione della disponibilità a lungo termine, l'approccio naive si rivela ancora più inefficiente nel caso reale, con spikes più significativi negli stessi contesti in cui anche il modello fatica a stabilizzare l'output.

5.4 Miglioramenti e sviluppi futuri

Gli sviluppi possibili per questo tipo di progetto sono molteplici, così come l'introduzione di metodologie più efficaci e strutturate per garantire una maggiore riproducibilità degli esperimenti. In particolare:

- Da un punto di vista metodologico, potrebbe essere utile adottare un approccio di Curriculum Learning più strutturato e tracciabile, prevedendo l'integrazione di strumenti di logging come MLflow al fine di monitorare più efficacemente il training, registrando parametri, metriche e versioni del modello per una migliore analisi delle performance e una gestione più ordinata degli esperimenti.
- Per rendere il modello più robusto si potrebbe intervenire sulla varietà delle sorgenti simulate. Introdurre molteplici funzioni di simulazione del vento permetterebbe all'agente di adattarsi a situazioni diverse, migliorando la sua capacità di generalizzazione e riducendo il rischio di sovradattarsi a scenari specifici del training.
- Attualmente, la gestione dell'idrogeno nel sistema si basa sul presupposto di un utilizzo immediato, mentre nella realtà l'iniezione di idrogeno avviene in un arco di tempo più lungo. Simulare questa transizione aiuterebbe a rendere il modello più fedele al comportamento del sistema reale.

- Si potrebbe aggiungere una maggiore variabilità nella domanda di energia, facendo oscillare lievemente il target e creando così un ambiente più dinamico, capace di mettere alla prova la flessibilità della strategia appresa dall'agente.
- Infine, il modello potrebbe essere migliorato espandendo la rete di gestione dell'energia, introducendo molteplici taniche di stoccaggio, elettrolizzatori e celle a combustibile. Un sistema più articolato obbligherebbe l'agente ad apprendere politiche sulla gestione del carico dei diversi componenti della rete. Questo non solo renderebbe il modello più complesso e realistico, ma lo preparerebbe anche a scenari di utilizzo più ampi e diversificati.